Rajesh Singh

Ext Res / Collaborators : None

No. of late day used in prev pset : 1
No. of late day used after including this : 1

Rajesh Singh CS1200 PSET-3.

1 (a) "Reporting Age Ranking" Solution :

(i) Recommendation: I would reccomend for Decennial Census, Bureau use "sorting and storing the sorted dataset" since data (id, bday) doesn't change for 10 years and hence once sorted (using runtime of O(nlogn)) the sorted data is stored and thus static datastructure ~~i Can Use~~ will be used efficienty for query rank since data is stored in sorted "bday" and rank can be evaluated based on this sorted (bday). function of "bday" will return "age rank"

(ii) How I would use datastructure: They would use "bday" as key, since age can be calculated using "bday" and id is irrelavant. Sorting via "bday" and assigning "rank" based on position of sorted key will return age rank.

(iii) Runtime: We are sorting (after getting new (id, bday) every 10 years. so MergeSort execution is O(nlogn) and storing sorted array is O(1) So, runtime would be O(nlogn)

 Rajash Singh CS1200 PSET-3.

1(b) "Daily Quartiles" Solution:

(i) Recommendation: I would reco that the Bureau use ~~AVL RBST~~ binary search tree (balanced) since ds need to be updated daily with insert/delete ~~of BRs~~ TR of ~~Binary~~ BST is O(nlogn) and search of 25th, 50th, 75th percentile of age ~~nnn~~ ~~RE~~ TR would be O(nlogn) Also BST is dynamic DS and ~~balan~~ balancing TR is just O(1).

(ii) How I would use my ds/algo: I would use "bday" (YYYYMMDD) as key and augment that # persin born at that day. Augmentation would help ~~to~~ calculate percentile for specific age.

(iii) Runtime: Calculating statistics require finding size(augmented value) at ~~25~~, ~~50~~ and ~~75~~ n/4, n/2, 3n/4 where population is n.
Search in balanced BST is $T_R = O(\log n)$
Also, Insert/update in BST $T_R = O(\log n)$

Rajesh Singh CS1200 PSET-3

(c) "Age lookups" solution:

(i) "Reccomendation": I would reco Bureau using "hash table" to store the data "key" using hash function

$h(id) = id \bmod m$

and will store (id, bday) in each bucket based on h_d.

(ii) "How I would use my data structure":
I would use hash function on id, assuming id as SSN. I would choos large # of buckets (m) like 100M to keep $\alpha = n/m$ very small and less collisim of key. Each bucket will store (id, bday)

~~For US, population is approx 350M~~
~~so α = 350/100 = 3.5~~
~~so, ra~~

(iii) Runtime: For hash table ~~run~~ ~~$T_R = O($~~

$T_R = O(1 + \frac{n}{m})$

assuming US population is 350M and $m = 100M$

so, avrge Avg. $T_R = O(1 + 3.5) \sim O(1)$
Also, delete/insert (update) is $\sim O(1)$ (Avg. T_R)
Worst case $T_R = O(n)$ - lookup, update.

1(d) "Daily Quartiles" Solution

(i) Recommendation : I would recommend using ~~Hash~~table to store (id, bday) in each bucket as in 1(c) and also binary tree (AVL) with augmented size of person's bday on specif day ~~bday as key~~. as in 1(b).
So, combination of 1(a) ~~for~~ and 1(b)

(ii) "How would i use ds/algo" : I will search bday for specific "id" in hash table. After I get bday, I can ~~insert~~ update AVL BST. For lookup of percentile, I can just use slon. as described in 1(b)

(iii) Runtime : For lookup $T_R = O(\log n)$ since population percentile can be directly published using BST.

For updates, require lookup + update.

so T_{Ravg} = Avg $O(1) + O(\log n)$
$\Rightarrow O(\log n)$

$T_{Rworst} = O(n) + O(\log n)$
$\Rightarrow O(n)$